

1. Set Up Your Tools	2
Visual Studio Code for Java development IDE.....	2
What is IDE.....	3
2. First java code in VSCode IDE.....	4
Create java project or standalone java extension file.....	4
Understand starting point of Java program	5
Compiling and linking.....	6
3. Primitive Data types, Variables.....	8
Variables.....	10
Reserved Words.....	11
4. Data representation	13
Bit, Byte, Convert Base 2 to Base 10 (Decimal).....	13
Text Representation (ASCII Code)	15
Range of all primitive datatype.....	16
Number system - Hexa-decimal number (Base-16).....	17
Conversion between number systems	18

Online tutorials for self-study

- Java for Beginners complete 2 hours tutorial: <https://www.youtube.com/watch?v=o33cQL6-2Sg&t=19s>

Udemy Learning Platform

- <https://www.udemy.com/courses/search/?src=ukw&q=java+tutorials>

1. Set Up Your Tools

- Before writing code, you need an environment to run it.
- Oracle Java Development Kit (JDK)
- Integrated Development Environment (IDE)

Visual Studio Code for Java development IDE

- Microsoft developed Visual Studio Code (VS Code) IDE.
- It is a fast, lightweight, and highly customizable code editor that has become an excellent option for Java programming. Seamlessly, work with other programming languages as well like Python and JavaScript.
- Alternatively, if you prefer not to install a JDK or use a local IDE, you can use online compilers like [Programiz](https://www.programiz.com/java-programming/online-compiler/) for quick programming. <https://www.programiz.com/java-programming/online-compiler/>

Getting Started Workflow

1. **Install the Software:** Use the unified "Coding Pack for Java" installer (available for Windows and macOS) to get **VS Code (IDE), the JDK, and crucial extensions simultaneously**. Download and install from: https://code.visualstudio.com/docs/java/java-tutorial#_setting-up-vs-code-for-java-development
2. **Generate a Project:** Press Ctrl+Shift+P (or Cmd+Shift+P on Mac) to open the Command Palette, type Java: Create Java Project, and choose No build tools for a clean starter setup.
3. **Write Code:** Build files inside the generated src folder using [integrated productivity shorthand snippets](#) like main or sysout.
4. **Execute:** Launch your app smoothly by clicking the built-in **Run** or **Debug** CodeLens buttons directly above your main class file.
5. **Getting started with VSCode** <https://code.visualstudio.com/docs/java/java-tutorial>

Follow the link to download and install Coding Pack for Java

1> Video link: <https://www.youtube.com/watch?v=tYYVeCB7Ef4>

Tutorial for using VS Code

<https://www.youtube.com/watch?v=bj6QSx22zXs>

What is IDE

An IDE, or Integrated Development Environment, is a software application that combines all the essential tools a developer needs to write, test, and debug code into a single program. Instead of using a separate text editor, compiler, and terminal, an IDE merges them into one interface to make programming faster and easier.

Core Components of an IDE

- **Source Code Editor:** A specialized text editor with features like auto-complete, syntax highlighting, and error checking.
- **Build Automation Tools:** Tools that automatically compile and build your raw code into an executable program.
- **Debugger:** A built-in diagnostic tool that lets you pause your program step-by-step to find and fix bugs.

IDE vs. Text Editor

- **Text Editors (e.g., Notepad++, basic VS Code):** Lightweight programs meant for typing raw text. They require you to manually install plug-ins or use external terminal commands to run your code.
- **IDEs (e.g., Eclipse, IntelliJ IDEA, Visual Studio):** Heavyweight applications that come completely pre-configured out of the box with compilers, linkers, and full execution environments.

2. First java code in VSCode IDE

Create java project or standalone java extension file

Standalone java File

How to create standalone java in VSCode: https://code.visualstudio.com/docs/java/java-tutorial#_creating-a-source-code-file

- Use this when you want to write a quick loop or test a piece of logic.
- You do not need any special configuration or folders.
- You cannot easily link it to other Java files.

Java Project

Follow link to create java project: <https://www.youtube.com/watch?v=tYYVeCB7Ef4>

- Use this when your program needs multiple files working together.
- It keeps your source code (src) separate from your compiled code (bin).
- It allows you to organize your files into **packages** (folders) as your code grows.

To create a Java project in VS Code, use the built-in Java project wizard.

Step-by-Step Setup

1. **Open Command Palette:** Press Ctrl + Shift + P (or Cmd + Shift + P on Mac).
2. **Launch Wizard:** Type **Java: Create Java Project** and select it from the list.
3. **Select Project Type:** Choose **No build tools** for a simple project (or select *Maven/Gradle* for advanced projects).
4. **Choose Location:** Select the folder where you want to save your project files.
5. **Name Project:** Input a name for your project folder and press **Enter**.

What Gets Created

VS Code automatically generates a standard folder structure for you.

- **src/:** The folder where you will write all your .java source code files.
- **lib/:** The folder to drop external libraries or .jar dependencies.
- **App.java:** A starter file containing a default "Hello, World!" program.

How to Run Your Project

- **One-Click Run:** Open App.java and click the **Run** button that appears directly above the main method.
- **Shortcut:** Press F5 on your keyboard to compile and debug the active file instantly

Key Differences

Feature	Standalone Java File	Java Project
Structure	A single <code>.java</code> file sitting alone.	A structured folder network (<code>src/</code> , <code>bin/</code> , <code>lib/</code>).
Use Case	Learning syntax, quick scripts, or testing code.	Building apps, software, or multi-screen programs.
External Libraries	Hard to add third-party packages or jars.	Built-in support to add external tools and packages.
Compilation	VS Code compiles only that single file.	VS Code compiles the entire application together.
Build Tools	None available.	Can use automation tools like Maven or Gradle.

Understand starting point of Java program

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Breakdown of the Code

- **public class Main:** Creates a container for your code. The class name must match your file name (e.g., Main.java).
- **public static void main:** The starting point (entry gate) where Java begins executing your program.
- **System.out.println():** The built-in command used to print text out to the screen

The **public static void main(String[] args)** line is the **exact entry point** for every Java application. When you run a Java program, the Java Virtual Machine (JVM) looks specifically for this exact sequence of words to start executing your code. If any part of it is missing or misspelled, your program will not start.

Compile the code: Type `javac Main.java` and press Enter to convert your code into machine-readable bytecode.

Execute the program: Type `java Main` and press Enter to see your message print to the console.
Or

Run from VSCode

The **Java Virtual Machine (JVM)** is an abstract computing machine that enables a computer to run Java programs. It acts as an intermediary runtime engine that sits between your Java code and the underlying computer hardware and operating system.

The Core Concept: "Write Once, Run Anywhere"

In traditional languages like C++, code is compiled directly into machine instructions tailored to a specific operating system (like Windows or macOS).

Java handles this differently:

1. You write Java source code (.java file).
2. The Java compiler (javac) converts your text into a universal intermediate form called **Bytecode** (.class file).
3. The **JVM** reads this bytecode and translates it on the fly into the specific machine code your current computer understands.

Because every operating system has its own specific version of the JVM, the exact same bytecode file can run on Windows, Mac, Linux, or a smartphone without any changes to the original source code

Compiling and linking

Compiling and **linking** are the two distinct steps required to transform your human-readable source code into a functional program that a computer can actually execute. While they happen automatically when you run a build command, they perform completely different jobs.

Compiling (Translation)

Compiling is the process of translating your source code text (like a .java or .cpp file) into a lower-level format that machines or virtual machines can understand.

- **What it does:** The compiler acts as a translator. It checks your code for syntax errors, typos, and improper rules. If your code is valid, it translates it.
- **The Output:**
 - In languages like **C/C++**, the compiler generates **Object Code** (individual .obj or .o files containing binary instructions).
 - In **Java**, the compiler (javac) generates universal **Bytecode** (individual .class files).
- **Key Characteristic:** The compiler only looks at **one file at a time**. It translates the file successfully even if that file references functions or modules that live in a completely separate file.

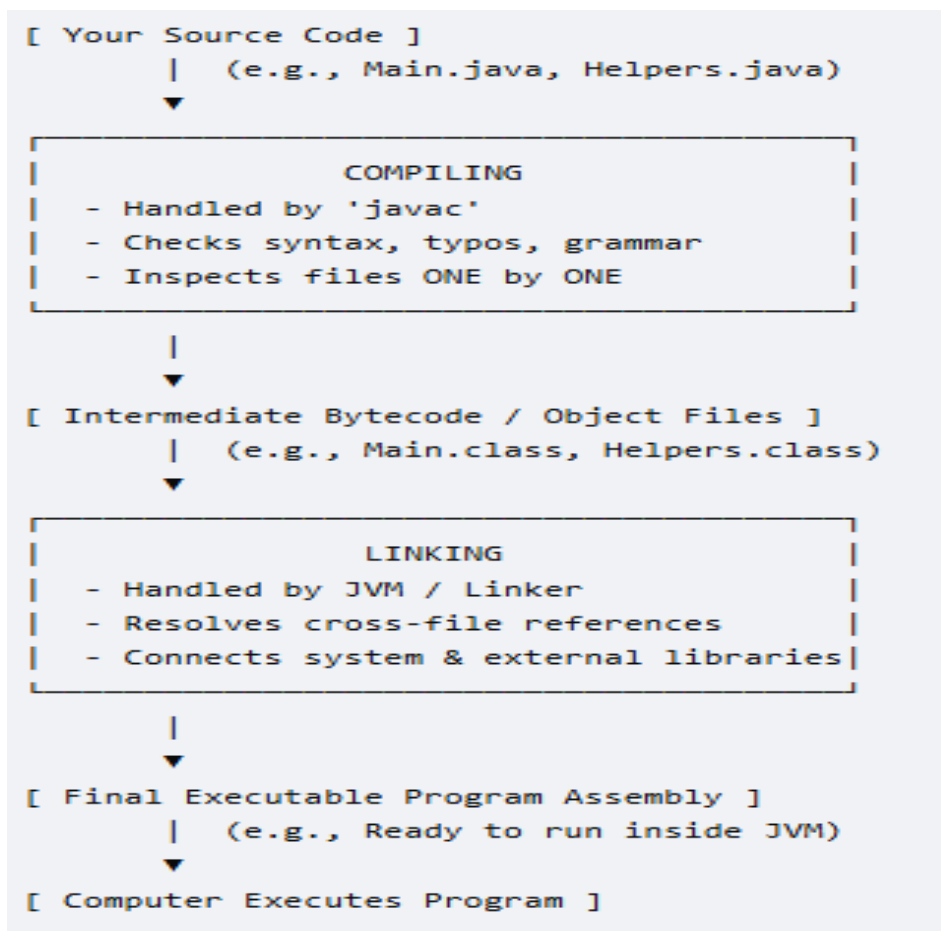
Linking (Assembly)

Linking is the process of gathering all the individual compiled files and external libraries, gluing them together, and building one final, cohesive executable file.

- **What it does:** The linker acts as an assembler. If File A calls a function that is written inside File B, or uses a built-in system library (like Java's System.out.println), the linker goes out, finds the exact memory locations of those missing pieces, and hooks them together.
- **The Output:**
 - In languages like **C/C++**, the linker outputs a single standalone executable file (like an .exe on Windows or a binary file on Mac/Linux).
 - In **Java**, true linking actually happens **dynamically at runtime** by the JVM's Class Loader, though you can manually use a tool like jar to pack your compiled .class files into a single .jar package.
- **Key Characteristic:** The linker looks at the **entire project**. If you try to use a function but forgot to write it or import its library, the compiler will pass it, but the linker will crash with an "unresolved external symbol" error.

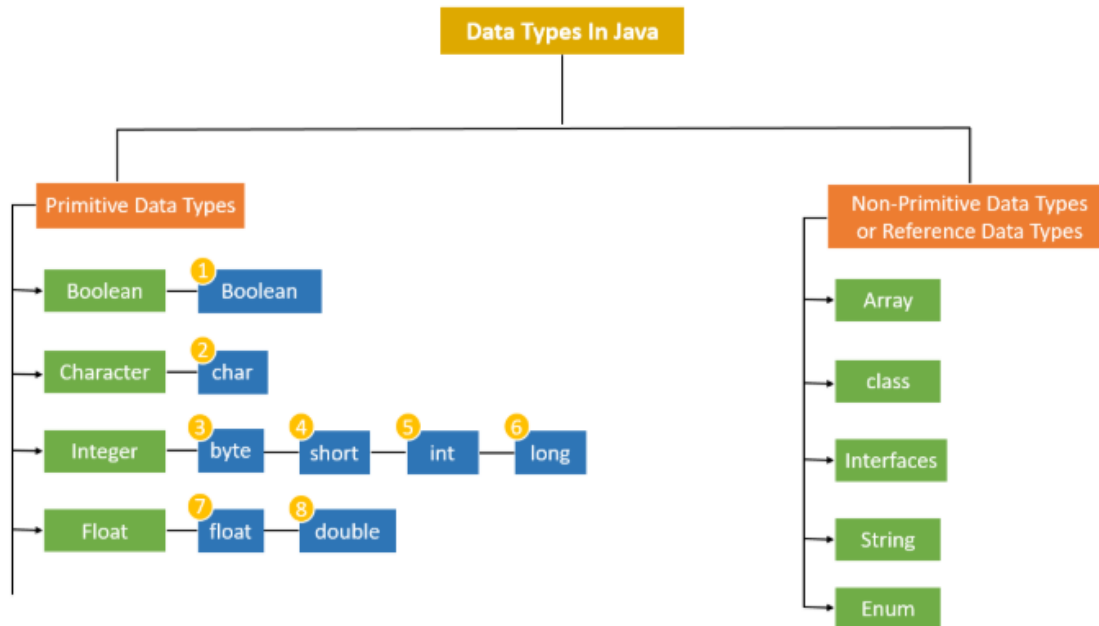
Walkthrough of the Diagram

1. **Source Code:** You write your human-readable Java logic across one or multiple files.
2. **Compiling:** The compiler translates your text files into bytecode (.class files). If you have a syntax error (like a missing semicolon), the process stops right here.
3. **Linking:** The system looks at all your separate .class files and any imported libraries. It binds them together so that when Main.class asks to use a method inside Helpers.class, the computer knows exactly where to jump in memory.
4. **Execution:** The computer safely runs your combined instructions.



3. Primitive Data types, Variables

Data types in Java specify the size and type of values that a variable can store. Java is a **strongly typed language**, meaning every variable must be declared with a specific data type before it can be used. Java data types are divided into two main categories: **Primitive** and **Non-Primitive**.



TYPE	DESCRIPTION	DEFAULT	SIZE	EXAMPLE LITERALS	RANGE OF VALUES
boolean	true or false	false	1 bit	true, false	true, false
byte	twos complement integer	0	8 bits	(none)	-128 to 127
char	unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\', '\n', '\r', '\t', '\f', '\b', '\u0000'	character representation of ASCII values 0 to 255
short	twos complement integer	0	16 bits	(none)	-32,768 to 32,767
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2	-2,147,483,648 to 2,147,483,647
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F	upto 7 decimal digits
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d	upto 16 decimal digits

Numeric Types (Integers): Used to store whole numbers without decimals.

Data Type	Size	Memory Range	Common Use Case
<code>byte</code>	1 byte (8 bits)	-128 to 127	Saving memory in large arrays
<code>short</code>	2 bytes (16 bits)	-32,768 to 32,767	Rare; used for memory constraints
<code>int</code>	4 bytes (32 bits)	-2.14 Billion to 2.14 Billion	Default choice for whole numbers
<code>long</code>	8 bytes (64 bits)	-9 Quintillion to 9 Quintillion	Large numbers (IDs, system time)

Note: When assigning a value to a long, append an L or l to the end of the number (e.g., long population = 8000000000L;)

Numeric Types (Floating-Point): Used to store fractional numbers containing decimals.

Data Type	Size	Precision	Common Use Case
<code>float</code>	4 bytes (32 bits)	6 to 7 decimal digits	Game graphics, saving memory
<code>double</code>	8 bytes (64 bits)	Up to 15 decimal digits	Default choice for math and decimals

⚠ **Note:** Java treats all decimal numbers as `double` by default. To declare a `float`, you must append an `F` or `f` to the value (e.g., `float price = 19.99f;`). ☹

Character Type: Used to store a single character.

Data Type	Size	Contents	Description
<code>char</code>	2 bytes (16 bits)	Single character or ASCII	Uses single quotes (<code>'A'</code> , <code>'3'</code> , <code>'\$'</code>). Uses Unicode system.

Boolean Type: Used to store conditional states.

Data Type	Size	Allowed Values	Common Use Case
<code>boolean</code>	1 bit (virtual)	<code>true</code> or <code>false</code>	Conditional flags (<code>if/else</code> logic)

Variables

A **variable** is a named storage container in a computer's memory used to hold data that a program can manipulate and change during execution. Think of it as a labeled box on a shelf; the label is the variable's name, and the item inside the box is its value.

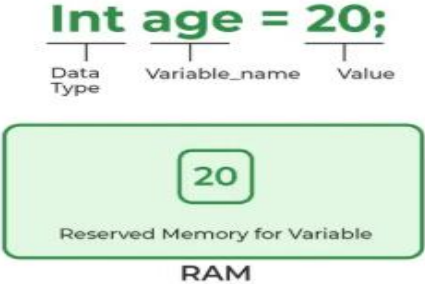
The Three Rules of Java Variables

Because Java is strongly typed, every variable must follow a strict three-part declaration syntax:

1. **Type:** What kind of data can fit inside the box (e.g., int, double, String).
2. **Name:** The identifier used to call upon that specific box in your code (e.g., age, price).
3. **Value:** The actual data stored inside the box (e.g., 25, 19.99).

Variable Syntax and Examples

You create a variable by **declaring** its type and name, and you populate it by **initializing** it with an assignment operator (=).

	<pre>// 1. Declaration only (Creating the empty box) int score; // 2. Initialization (Putting a value into the box) score = 10; // 3. Combined Declaration and Initialization (Most common) String username = "Alex"; // 4. Changing the value later (Overwriting the box contents) score = 15;</pre>
--	---

Variable Scopes in Java

Where you declare a variable determines where it can be used inside your code:

- **Local Variables:** Declared inside a method (like main). They only exist while that specific method is running and cannot be accessed from outside it.
- **Instance Variables:** Declared inside a class but outside any methods. They belong to objects created from the class.
- **Static Variables:** Declared with the static keyword inside a class. They belong to the class itself and are shared globally across all instances.

Rules for Naming Variables (Identifiers)

- Names are **case-sensitive** (age and Age are completely different boxes).
- Names must start with a letter, a dollar sign (\$), or an underscore (_). They cannot start with a number.
- Names cannot contain spaces.
- Names cannot use Java reserved keywords (you cannot name a variable public or class).
- Java convention uses **camelCase** for multi-word names

Reserved Words

Reserved words (also known as **keywords**) are predefined words that have a special, built-in meaning to the Java compiler. Because the language reserves these words for its own structural rules, logic, and commands, you **cannot** use them as names for your variables, classes, methods, or any other identifiers.

Java has over 50 reserved words. They are always written in lowercase.

Categories of Common Reserved Words

1. Data Types & Values

Words used to declare variable types or literal states.

- **int, double, char, boolean, long, float, byte, short:** Define primitive data types.
- **true, false:** Literal boolean values.
- **null:** Represents an empty or non-existent reference.

2. Access Modifiers

Words that control which parts of your program can see or use a class, method, or variable.

- **public:** Accessible from anywhere in your project.
- **private:** Accessible only inside the specific class where it was created.
- **protected:** Accessible within the same package or by child classes.

3. Flow Control & Logic

Words that dictate the order in which your code runs.

- **if, else:** Used to make conditional decisions.
- **switch, case, default:** Used for multi-way branching selections.
- **for, while, do:** Used to create loops that repeat code blocks.
- **break:** Exits a loop or switch block immediately.
- **continue:** Skips the rest of the current loop iteration and moves to the next one.
- **return:** Exits a method and optionally sends back a value.

4. Program Structure & Modifiers

Words used to build and organize the layout of your application.

- **class:** Declares a new object blueprint.
- **interface:** Declares a structural contract for classes.
- **package:** Groups related classes together into a folder system.
- **import:** Brings external classes or packages into your current code file.
- **static:** Attaches a method or variable directly to the class rather than an object instance.
- **void:** Tells a method that it will not return any data.
- **final:** Makes a variable constant (unchangeable) or prevents a class from being inherited.

✗ Invalid vs. ✓ Valid Variable Names

java

// INVALID CODE - Will cause a compiler error:

`int public = 25;` *// Error: 'public' is a reserved word*

`double class = 5.99;` *// Error: 'class' is a reserved word*

`boolean void = true;` *// Error: 'void' is a reserved word*

// VALID CODE - Safe to use:

`int publicScore = 25;` *// OK: Combines keyword with another word*

`double classPrice = 5.99;` *// OK: Modified name*

`boolean isValid = true;` *// OK: Plain English word (not reserved)*

4. Data representation

1> <https://www.techtarget.com/searchstorage/definition/byte>

2> <https://web.stanford.edu/class/cs101/bits-bytes.html>

Video: <https://www.youtube.com/watch?v=KLR5Y2uBYng>

Bit, Byte, Convert Base 2 to Base 10 (Decimal)

- Computers are electronic devices that store voltages. These devices are either ON or OFF. Thus only 2 voltages are possible, 1 for ON and 0 for OFF. Thus, computer stores information (data) as two symbols ie 0 and 1.
- Data (0 and 1) is a physical concept, a raw sequence of bits and bytes stored on a computer. Data has no inherent meaning and must be interpreted by whoever is reading it. Computers store data using a sequence of 1's and 0's. Later, they read back the 1's and 0's and try to recreate the abstract concept from the raw data. Depending on the assumptions made, the 1's and 0's can mean very different things.
- What is a Bit? (The Single Switch)**
 - A **bit** (short for **binary digit**) is the smallest piece of information a computer can understand.
 - A computer's brain is made of billions of microscopic electronic switches. Because a switch can only be in two positions, a bit can only have one of two values: **0** = Off (Low voltage / False) **1** = On (High voltage / True)
 - A single bit cannot tell you much. It can only answer simple "yes or no" questions.
- What is a Byte? (The Character Builder)**
 - Because one bit is too small to be useful on its own, computers bundle them into groups of eight. With 8 bits combined, you can create **256 different combinations** of zeros and ones (from 00000000 to 11111111)
 - A **byte** is a group of **8 bits** working together. Here is how a single byte is structured and represented. A byte consists of 8 slots. Each slot is a bit that holds a value of either 0 or 1.

	1 Byte							
	Upper Nibble				Lower Nibble			
Bit position	Bit -7	Bit-6	Bit-5	Bit-4	Bit-3	Bit-2	Bit-1	Bit-0
Binary number	MSB							LSB
	1	0	0	1	0	1	1	0
Base-2 Place value	1×2^7 = 128	1×2^6 = 64	1×2^5 = 32	1×2^4 = 16	1×2^3 = 8	1×2^2 = 4	1×2^1 = 2	1×2^0 = 1
Convert Base-2 to Base-10(decimal)	1×2^7 = 128	0×2^6 = 0	0×2^5 = 0	1×2^4 = 16	0×2^3 = 0	1×2^2 = 4	1×2^1 = 2	0×2^0 = 0
Value(decimal)	150							

- Computers use a Base-2 (Binary) place value system, Meaning each slot's value doubles from right to left, starting at 1.

Bit Position	8	7	6	5	4	3	2	1
Place Value	128	64	32	16	8	4	2	1
Bit State	0	1	0	0	0	0	0	1

- Bits are numbered from right-to-left. Bit 0 is the rightmost and the smallest; bit 7 is leftmost and largest. The "leftmost" bit in a byte is the biggest.
- Bit numbering is bit positions in a binary number. The bit number starts with zero and is incremented by one for each subsequent bit position. The value of the number is calculated by using exponentiation with the bit number and a base of 2.

- These bits are grouped as

- Nibble (4 bits): A nibble is a sequence of 4 bits

- 8 bits makes 1-byte

8-bits = 1-byte = 1101 0010

- 16 bits makes 1-word

16bits = 1 word 0000 0010 1101 0010

- 32 bits makes 1-double word

32bits = 1 double word 0000 0000 0000 0010 1101 0010

- Example

The binary sequence 00001001 is decimal number 9.

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
binary sequence	0	0	0	0	1	0	0	1
multiply by place holder.	0×2^7	0×2^6	0×2^5	0×2^4	1×2^3	0×2^2	0×2^1	1×2^0
	0	0	0	0	8	0	0	1
Decimal number	9							

- Range of 1 byte variable.

A byte can represent any whole number from **0 to 255**.

00000000 = 0

01000001 = 65

11111111 = 255 (All switches on: $128+64+32+16+8+4+2+1$)

Text Representation (ASCII Code)

Computers use a translation chart called **ASCII** (American Standard Code for Information Interchange) to map numbers to keyboard characters.

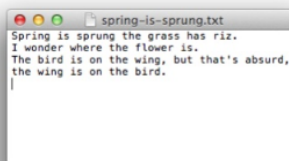
- The byte 01000001 equals 65.
- On the ASCII chart, 65 is the capital letter "A".
- The word "HI" requires two bytes: 01001000 (H) and 01001001 (I)
- One byte works well for individual characters.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Typing, Bytes, and You

- Each letter is stored in a byte, as below
- 100 typed letters takes up 100 bytes
- When you send, say, a text message, the numbers are sent
- Text is quite compact, using few bytes, compared to images etc.



Underlying bytes in RAM

S	p	r	i	...
83	112	114	105	

Visualizing the Scale

To see how this grows into the files you use every day, look at how bytes multiply:

- **1 Bit:** A single light switch (On or Off).
- **1 Byte (8 bits):** A single letter (e.g., "H").
- **1 Kilobyte (KB, ~1,000 bytes):** A short paragraph of text.
- **1 Megabyte (MB, ~1 million bytes):** A 3-minute MP3 song or a high-quality photo.
- **1 Gigabyte (GB, ~1 billion bytes):** A 1-hour streaming movie

Range of all primitive datatype

- Integers** are typically stored with either 4 or 8 bytes
 4 bytes can store numbers between -2147483648 and 2147483647
 8 bytes can store numbers between -9223372036854775808 and 9223372036854775807
- Adding in binary is just like normal addition with carrying
 But when you run out of bits you can't carry anymore
 Leftmost bit indicates sign, so carrying to the leftmost bit changes a number from positive to negative.
 So adding 1 to 2147483647 goes to -2147483648!
 Called **Integer Overflow**
 - byte is the primitive type
 - Byte is the corresponding wrapper class
 - Byte.SIZE gives the number of bits in a byte
 - Byte.MIN_VALUE and Byte.MAX_VALUE give the minimum and maximum values a primitive byte can hold

```

1 public class DataTypesDemo {
2     public static void demonstrate() {
3         int a = 7;
4         int b = 5;
5         int intSum = a + b;
6
7         float x = 3.5f;
8         float y = 2.2f;
9         float floatSum = x + y;
10
11         System.out.println("Integer addition: " + a + " + " + b + " = " + intSum);
12         System.out.println("Float addition: " + x + " + " + y + " = " + floatSum);
13         System.out.println();
14         displayPrimitiveSizesAndRanges();
15     }
16
17     private static void displayPrimitiveSizesAndRanges() {
18         System.out.println(x: "Primitive type sizes and ranges:");
19         System.out.println("byte:   size = " + Byte.SIZE + " bits, range = " + Byte.MIN_VALUE + " to " + Byte.MAX_VALUE);
20         System.out.println("short:  size = " + Short.SIZE + " bits, range = " + Short.MIN_VALUE + " to " + Short.MAX_VALUE);
21         System.out.println("int:    size = " + Integer.SIZE + " bits, range = " + Integer.MIN_VALUE + " to " + Integer.MAX_VALUE);
22         System.out.println("long:   size = " + Long.SIZE + " bits, range = " + Long.MIN_VALUE + " to " + Long.MAX_VALUE);
23         System.out.println("float:  size = " + Float.SIZE + " bits, range = " + Float.MIN_VALUE + " to " + Float.MAX_VALUE);
24         System.out.println("double: size = " + Double.SIZE + " bits, range = " + Double.MIN_VALUE + " to " + Double.MAX_VALUE);
25         System.out.println("char:   size = " + Character.SIZE + " bits, range = " + (int) Character.MIN_VALUE + " to " + (int) Character.MAX_VALUE);
26         System.out.println(x: "boolean: size = 1 bit (logical), values = true or false");
27     }
28 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Float addition: 3.5 + 2.2 = 5.7

Primitive type sizes and ranges:

```

byte:   size = 8 bits, range = -128 to 127
short:  size = 16 bits, range = -32768 to 32767
int:    size = 32 bits, range = -2147483648 to 2147483647
long:   size = 64 bits, range = -9223372036854775808 to 9223372036854775807
float:  size = 32 bits, range = 1.4E-45 to 3.4028235E38
double: size = 64 bits, range = 4.9E-324 to 1.7976931348623157E308
char:   size = 16 bits, range = 0 to 65535
boolean: size = 1 bit (logical), values = true or false

```

Number system - Hexa-decimal number (Base-16)

Any numerical value (number) can be represented in any of the following number system.

Number Base	Number system	
Base 2	binary	The common representation of data in computer is a string of bits, using the binary numeral system (Base-2).
Base 8	octal	
Base 10	decimal	The most familiar representation of numbers is in base 10 systems. Eg 12345.
Base 16	hexadecimal	Binary strings (01000001) are long and hard for human programmers to read. To simplify this, programmers split a byte in half (two 4-bit sections called nibbles) and convert them into Hexadecimal (Base-16). It is convenient shorthand notation to talk about binary numbers. Hexadecimal is also commonly used to represent computer memory addresses.

Hexadecimal is a number system that uses 16 symbols to represent numbers.

- Hexadecimal uses the symbols 0–9 to represent the values 0–9.
- It uses the letters A–F to represent the values 10–15. In hexadecimal, when you reach the decimal number 10, you switch to letters A through F because a single digit can hold any value up to 15.
- Each digit represents a decimal value. For example, D is equal to 13 in base 10.

Decimal (Base 10)	Hexadecimal (Base 16)	4-Bit Binary Value
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Conversion between number systems

<u>Decimal to Hexa-decimal conversion</u>									<u>Hex to decimal conversion</u>				
Identify the group of 4 bits each. Fill the partial group with 0's on outside. Then replace each group with the corresponding hex digit.									hexadecimal number (Base-16) 0x2AF3 is equal to 10,995 decimals (base-10).				
0011	1010	0101	0110	.	1110	0010	1111	1000	Base-16 to base-10	2	A	F	3
3	A	5	6	.	E	2	F	8		2×16^3	$A \times 16^2$	$F \times 16^1$	3×16^0
										2×16^3	10×16^2	15×16^1	3×16^0
										2x4096 =8192	10x256 =2560	15x16 =240	3*1 =3
									Base-10	10,995			

8-Bits (1byte)	1	1	1	1	1	1	1	1
Base -2 place holder	$1 \times 2^7 =$ (128)	$1 \times 2^6 =$ (64)	$1 \times 2^5 =$ (32)	$1 \times 2^4 =$ (16)	$1 \times 2^3 =$ (8)	$1 \times 2^2 =$ (4)	$1 \times 2^1 =$ (2)	$1 \times 2^0 =$ (1)
Decimal value	255							
Hexadecimal value	0xFF							